Chapter 2

Background

2.1 Reinforcement Learning

Reinforcement Learning (RL) is a branch of Machine Learning (ML) concerned with learning how to make sequential decisions through interaction with an environment. Within the Machine Learning area, a general distinction is made between supervised, unsupervised, and Reinforcement Learning.
In *supervised learning*, an algorithm is trained on a labeled dataset containing examples, with the objective of learning a mapping that generalizes to samples not belonging to it. On the other hand, *unsupervised learning* works on unlabeled data and aims to identify underlying structures and patterns within the available observations [14].
Reinforcement Learning differs from both paradigms because the learner is not provided with explicit examples of the correct action to perform. Instead, an agent interacts with an (initially) unknown environment, selects actions, observes their consequences, and receives a reward signal that provides feedback about its behavior. Therefore, the goal of the agent is to learn a policy that maximizes the expected cumulative reward over time.
This type of interaction describes a *sequential decision-making problem*. Unlike prediction problems in which individual outputs can often be considered independently, in RL the consequences of a decision may extend beyond the immediate transition. An action selected at a given time may influence the state reached by the agent, the actions that will subsequently become available, and the rewards that can be obtained in the future. So, the quality of a decision cannot generally be evaluated only in terms of its immediate outcome, but must also account for its long-term consequences. In fact, in RL we often refer to trajectories (sequences of states and actions) generated by the agent's behavior during its interaction with the environment.
RL can therefore be viewed as a framework for sequential decision-making under uncertainty, where an agent must learn how its actions influence both immediate and future outcomes. To reason formally about these interactions, a mathematical model of the decision process is required. The standard framework adopted for this purpose is the Markov Decision Process (MDP), which provides a formal description of states, actions, transition dynamics, and rewards. MDPs are introduced in the following

section and will be the basis for the reinforcement learning methods discussed throughout this chapter.

2.1.1 Markov Decision Processes

The mathematical framework adopted for modeling sequential decision-making problems is the Markov Decision Process (MDP). An MPD is defined as a tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition probability function, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor [14].
At each discrete time step t, the environment is in a state $S_t \in \mathcal{S}$. Based on this state, the agent selects an action $A_t \in \mathcal{A}$. The execution of the action triggers the environment to transition into a new state S_{t+1} and produces a scalar reward R_{t+1}.

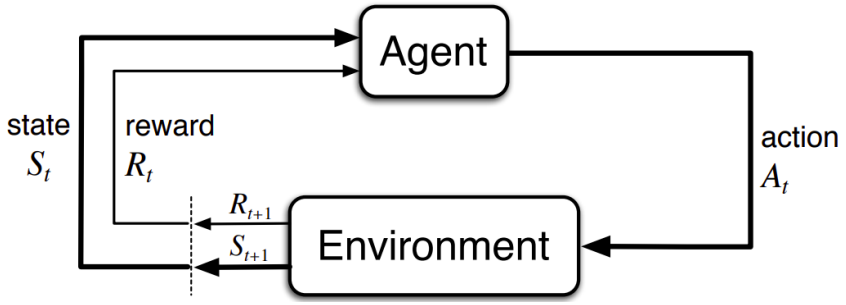

Figure 2.1. Interaction between the agent and the environment in an MDP [14].

The transition dynamics are characterized by

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{2.2}$$

which denotes the probability of reaching state s' after executing action a in state s. Looking at the P definition, it is immediate to notice that s' strictly depends only on the immediate previous state s. Therefore, with this definition, we are implicitly assuming the so-called *Markov property*: conditioned on the current state and action, the distribution of the next state is independent of the preceding interaction history. Formally, it can be expressed as follow:

$$\Pr(s' \mid s, a, S_{t-1}, A_{t-1}, \ldots, S_0) = \Pr(s' \mid s, a) \tag{2.3}$$

The current state must contain all the information relevant to predicting the future evolution of the process [14]. If additional information from the interaction history is required, the chosen state representation is *not Markovian* with respect to the problem under consideration.
The reward function assigns a numerical signal to the agent's interaction with the environment. The most general formulation corresponds to the function $R(s, a, s')$ which provides immediate feedback, whereas the agent's objective generally depends on rewards accumulated over multiple time steps. The distinction between immediate and long-term consequences is therefore central to sequential decision-making.

The discount factor γ determines the relative importance of future rewards. Values close to zero emphasize immediate outcomes, whereas values close to one assign greater importance to rewards received further in the future. The accumulation of discounted rewards will be formalized through the concept of return in the subsection on policies, returns, and value functions.
A sequence generated through interaction between the agent and the environment can be written as

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots \tag{2.4}$$

and is referred to as a *trajectory*. The probability of a trajectory is determined jointly by the transition dynamics of the MDP and the actions chosen by the agent based on its own *policy*.

2.1.2 Policies, Returns, and Value Functions

A *policy* specifies how the agent selects actions in each state. A stochastic policy π assigns a probability to every action $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s) \tag{2.5}$$

For every state, these probabilities satisfy $\pi(a \mid s) \geq 0$ and $\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$. A deterministic policy is the special case in which one action is selected with probability one and can therefore be represented as a mapping $\pi : \mathcal{S} \to \mathcal{A}$.
The immediate reward obtained from a single transition is generally insufficient to evaluate an action, since its consequences may affect rewards received many steps later. The long-term outcome from time step t is represented by the *return*, defined as the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{2.6}$$

In episodic tasks, this sum terminates when a terminal state is reached. The discount factor γ controls the relative importance of immediate and delayed rewards. For continuing tasks with bounded rewards, choosing $\gamma < 1$ also ensures that the infinite sum remains finite.
Value functions measure the expected return obtained when the agent follows a given policy. The *state-value function* evaluates a state, whereas the *action-value function* evaluates the selection of a particular action in that state:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \tag{2.7}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{2.8}$$

The expectation accounts for both the actions selected according to π and the stochastic transitions of the environment. The state-value and action-value functions are related by

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s, a) \tag{2.9}$$

An optimal policy π^* maximizes the expected return from every state. The corresponding optimal value functions are

$$V^*(s) = \max_\pi V^\pi(s) \tag{2.10}$$

$$Q^*(s,a) = \max_\pi Q^\pi(s,a) \tag{2.11}$$

Once Q^* is known, an optimal deterministic policy can be obtained by selecting an action with maximum value:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} Q^*(s,a) \tag{2.12}$$

Policies and value functions therefore provide the principal quantities used to describe and compare decision-making strategies. Their recursive structure is captured by the Bellman equations [14].

2.1.3 Bellman Equations and Value Iteration

The return in Equation (2.6) can be decomposed into the immediate reward and the discounted return from the next time step:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.13}$$

This recursive relation leads to the *Bellman expectation equation*. For a policy π, the value of a state equals the expected immediate reward plus the discounted value of the successor state:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right] \tag{2.14}$$

The corresponding equation for the action-value function is

$$Q^\pi(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^\pi(s',a') \right] \tag{2.15}$$

These equations express a consistency condition: the value assigned to the current state or action must agree with the values assigned to its possible successors. For an optimal policy, the expectation over the next action is replaced by a maximization, yielding the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s') \right] \tag{2.16}$$

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a') \right] \tag{2.17}$$

When the transition and reward models are known, the optimal state-value function can be computed through *value iteration*. Starting from an arbitrary estimate V_0, the Bellman optimality backup is applied repeatedly:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right] \tag{2.18}$$

For a finite discounted MDP, the sequence of estimates converges to V^*. An optimal policy can then be extracted by acting greedily with respect to the converged values:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2.19}$$

Value iteration is a model-based planning method because it requires explicit knowledge of P and R. In RL, these quantities are often unknown, motivating model-free methods like Q-learning [14].

2.1.4 Goal-Oriented Tasks and Sparse Rewards

Many RL problems are naturally formulated in terms of achieving a specific objective. In these scenarios, the agent receives a reward signal only when it reaches the final goal, and therefore it is not guided through a reward signal that continuously assesses every intermediate action. Such problems can be modeled as goal-oriented MDPs, in which the task objective is explicitly associated with a goal condition.

Let g denote a goal and let $\mathcal{G}_g \subseteq \mathcal{S}$ be the set of states that satisfy it. A simple sparse formulation assigns a positive reward only when a transition reaches a goal state:

$$R_g(s, a, s') = \begin{cases} r_g, & \text{if } s' \in \mathcal{G}_g, \\ 0, & \text{otherwise} \end{cases} \tag{2.20}$$

where $r_g > 0$ is the reward associated with successful goal achievement.
Depending on the setting, reaching a goal state may also terminate the episode.

A reward function is described as *sparse* when the agent receives informative feedback only on a small fraction of the transitions it experiences. On the other hand, when the agent receives a high frequency reward signal from the environment, we refer to *dense* reward function.

A notable sparse-reward setting is the *goal MDP*: an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a *goal MDP* if and only if there exists a set $\mathcal{G} \subseteq \mathcal{S}$ of goal states for which the reward function satisfies

$$R(s, a, s') = \begin{cases} 1, & \text{if } s \notin \mathcal{G} \text{ and } s' \in \mathcal{G}, \\ 0, & \text{otherwise} \end{cases} \tag{2.21}$$

In addition, goal states have zero optimal continuation value:

$$V^*(s) = 0, \qquad \forall s \in \mathcal{G} \tag{2.22}$$

Under these conditions, the agent receives a unit reward only when it enters the goal set from a non-goal state. Once a goal state has been reached, no further return can be accumulated from that state.

Sparse rewards provide a direct representation of the task objective because they reward behavior that actually achieves the final goal. However, they also make learning more difficult [14, 6]. Before encountering a successful trajectory, the agent may receive the same reward for many different actions and states, with no direct indication of whether its behavior is moving toward or away from the objective. This difficulty is closely related to exploration. Without informative intermediate feedback, the agent must discover successful behavior largely through interaction with the environment. If the goal can be reached only after a long sequence of appropriate decisions, the probability of finding a successful trajectory without the reward signal's guidance may be very low.
Sparse-reward problems will be central into the framework proposed in Chapter 3.

2.1.5 Value-Based Methods

Value-based reinforcement learning methods compute an optimal policy by estimating value functions rather than representing the policy directly. In particular, action-value methods learn the expected return associated with each state-action pair and derive a policy by selecting actions adopting a greedy strategy with respect to these estimates.

Q-learning. Q-learning is a model-free, off-policy algorithm that estimates the optimal action-value function directly from observed transitions [16]. After observing the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it applies the temporal-difference update

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right] \tag{2.23}$$

where α is the learning rate hyper-parameter. The maximization defines a greedy target policy, while experience can be collected through an exploratory behavior policy such as ϵ-greedy. Tabular Q-learning is effective for small discrete MDPs, but it becomes impractical when the state space is large because it maintains a separate estimate for every (s, a) pair [14].

Deep Q-Networks. Deep Q-Networks (DQN) address this limitation by approximating the action-value function with a neural network $Q(s, a; \theta)$. To improve training stability, DQN stores transitions in a replay memory and learns from randomly sampled mini-batches. It also maintains a target network with parameters θ^-, which are periodically synchronized with the online parameters θ. For a transition (s, a, r, s', d), the target is

$$y^{\mathrm{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \theta^-) \tag{2.24}$$

where d indicates whether the transition is terminal. The online network is trained by minimizing the squared difference between this target and $Q(s, a; \theta)$ [9].

Double Deep Q-Networks. The maximization in the DQN target can lead to overestimated action values because the same estimates are used both to select and evaluate the next action. Double Deep Q-Networks (DDQN) reduce this bias by using the online network for action selection and the target network for action evaluation:

$$y^{\text{DDQN}} = r + \gamma(1-d)Q\left(s', \arg\max_{a' \in \mathcal{A}} Q(s', a'; \theta); \theta^-\right) \tag{2.25}$$

DDQN retains the replay memory, target network, and optimization procedure of DQN while changing only the construction of the temporal-difference target. This generally produces more reliable value estimates and is the value-based learning algorithm adopted in the proposed framework [15].

2.2 Temporally Extended Tasks

In several situations, a problem specification cannot be expressed as an objective that depends only on the last state of the environment. Specifically, in this scenarios, a successful behavior could depend on the order in which events occur, on whether certain conditions have already been satisfied, or on whether specific events are eventually reached during the execution.
For instance, consider a task requiring an agent to first visit a region (A) and subsequently reach a region (B). Reaching (B) cannot be considered sufficient to determine task completion unless we have not already reached region (A) in the past. Therefore, two trajectories ending in the region (B) may correspond to different levels of progress with respect to the task.
This types of objective specifications are commonly referred to as *temporally extended tasks*, since their satisfaction is defined over sequences of states or observations rather than on individual states only. When the information required to evaluate the task is not completely represented by the current environment state, the task introduces a form of *non-Markovian* dependence on the interaction history (the *Markov property* is violated).
Representing such objectives requires a formalism capable of expressing temporal relationships between events. In this thesis, we choose to express these goal specifications trough Linear Temporal Logic on finite traces (LTLf), a logical language which allows temporal properties to be specified over finite executions. Then, LTLf specifications can in turn be converted in a deterministic finite automata (DFA), whose states provide a way to monitor the task progress.

2.2.1 Markovian and Non-Markovian Tasks

As introduced in Section 2.1.1, the Markov property requires the current state to contain all the information needed to characterize the future evolution of the environment [14].
In a MDP, the probability distribution $P(s' \mid s, a)$ of the next state depends only on the current state and action, and not on the complete interaction history.

A similar distinction can be made with respect to the task or reward structure.
In a *Markovian task*, the relevance of a transition can be determined from the current state, action, and successor state. So, under this assumption, a reward function can be expressed as $R(s, a, s')$.

However, there are some objectives that cannot be evaluated using only the current transition. In particular, their satisfaction depends on events that have occurred earlier in the execution. Let

$$h_t = s_0, a_0, s_1, a_1, \ldots, s_t \tag{2.26}$$

denote the interaction history up to time t. When the reward or the task satisfaction depend on h_t, we are referring to a *Non-Markovian task specification.*
This distinction is crucial because the environment itself may still satisfy the Markov property even when the task specification does not. In other words, non-Markovianity may arise from the specification of the objective rather than from the dynamics of the environment. The physical state can therefore be sufficient to predict the evolution of the environment while being insufficient to determine the progress made toward the task.

This concept is formalized by the *Non-Markovian Reward Decision Process* (NMRDP) definition introduced in [3].
An NMRDP is defined as a tuple:

$$\mathcal{M}_{\mathrm{NM}} = \left\langle \mathcal{S}, \mathcal{A}, P, \overline{R}, \gamma \right\rangle \tag{2.27}$$

where $\mathcal{S}$, $\mathcal{A}$, P, and γ have the same meaning as in an MDP, while the reward function is defined over finite sequences of state-action pairs:

$$\overline{R} : (\mathcal{S} \times \mathcal{A})^* \to \mathbb{R} \tag{2.28}$$

Let

$$\tau = \langle (s_0, a_0), (s_1, a_1), \ldots, (s_{n-1}, a_{n-1}) \rangle \tag{2.29}$$

be a finite trace, and let $\tau_{\leq i}$ denote its prefix containing the first i state-action pairs. The value of τ is defined as

$$V(\tau) = \sum_{i=1}^{|\tau|} \gamma^{i-1} \overline{R}(\tau_{\leq i}) \tag{2.30}$$

Unlike a Markovian policy, whose action depends only on the current state, a policy for an NMRDP may depend on the complete sequence of states observed so far:

$$\overline{\pi} : \mathcal{S}^* \to \mathcal{A} \tag{2.31}$$

Therefore, both the reward and the policy may depend on the interaction history rather than exclusively on the current state.

2.2.2 Linear Temporal Logic on Finite Traces

Linear Temporal Logic (LTL) [12] is a formal language used to describe properties of sequences that evolve over time. Unlike propositional logic, which evaluates formulas with respect to a single state (the *interpretation*), temporal logic allows the specification of relationships between events occurring at different points of an execution.
LTL is interpreted over infinite traces. However, in episodic RL, interactions agent-environment are naturally represented by finite sequences, since an episode terminates after a finite number of steps. Therefore, in this thesis, we mainly consider LTLf which is a variant of LTL evaluated on finite traces [7].

Let P be a finite set of atomic propositions describing relevant properties of the environment. At each time step, a subset of these propositions is true. A finite trace over P can therefore be represented as

$$\rho = \rho_0, \rho_1, \ldots, \rho_{n-1} \tag{2.32}$$

where

$$\rho_i \in 2^P \tag{2.33}$$

denotes the set of atomic propositions that hold at position i.
An LTLf formula is built from atomic propositions using boolean and temporal operators. Formally,

$$\varphi ::= A \mid (\neg\varphi) \mid (\varphi_1 \wedge \varphi_2) \mid (\circ\varphi) \mid (\varphi_1 \, U \, \varphi_2) \mid (\Diamond\varphi) \mid (\Box\varphi) \tag{2.34}$$

where $A \in P$, $\circ$ is the *next* operator, U is the *until* operator, $\Diamond$ is the *eventually* operator and $\Box$ it the *globally* operator.
The satisfaction of an LTLf formula is defined with respect to a position of a finite trace. For an atomic proposition p,

$$\rho, i \models p \quad \Longleftrightarrow \quad p \in \rho_i \tag{2.35}$$

A finite trace ρ satisfies an LTLf formula φ, written as

$$\rho \models \varphi \tag{2.36}$$

when the formula is satisfied at the initial position of the trace. Hence, every LTLf formula defines a set of finite traces that satisfy the corresponding temporal specification.

2.2.3 Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) [2] is a finite-state machine useful to recognize languages defined over a finite alphabet. Formally, a DFA is defined as a tuple

$$\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle \tag{2.37}$$

where Q is a finite set of states, Σ is a finite input alphabet, δ is the transition function

$$\delta : Q \times \Sigma \to Q \tag{2.38}$$

$q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.
The automaton processes an input word one symbol at a time. Since the transition function is deterministic, for every state $q \in Q$ and input symbol $\sigma \in \Sigma$, there exists a unique successor state

$$q' = \delta(q, \sigma) \tag{2.39}$$

Given a finite word

$$w = \sigma_0 \sigma_1 \dots \sigma_{n-1} \in \Sigma^* \tag{2.40}$$

the execution, or run, of the automaton over w is the sequence of states

$$q_0, q_1, \dots, q_n \tag{2.41}$$

such that

$$q_{i+1} = \delta(q_i, \sigma_i) \qquad 0 \leq i < n \tag{2.42}$$

The word w is accepted by the automaton if the state reached after processing the complete word belongs to the accepting set:

$$q_n \in F \tag{2.43}$$

The set of all finite words accepted by $\mathcal{A}$ defines the language recognized by the automaton, and it is denoted by

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \mathcal{A} \text{ accepts } w\} \tag{2.44}$$

2.2.4 From LTLf Specifications to DFA

A LTLf formula can be represented trough an equivalent DFA. As described in [7], this property is achieved thanks to the fact that every LTLf formula defines a regular language over finite traces.
Formally, let's consider the LTLf formula φ defined over a set P of atomic propositions. We can define the DFA $\mathcal{A}_\varphi = \langle Q, 2^P, \delta, q_0, F \rangle$ in such a way that for every finite trace ρ holds:

$$\rho \models \varphi \quad \iff \quad \rho \in L(\mathcal{A}_\varphi) \tag{2.45}$$

Therefore, the LTLf formula and the corresponding DFA provide two different representations of the same temporal specification. In particular, the former provides a declaratively way to describe a temporal specification, instead the latter provides an operational representation useful for the integration of the formula in an algorithm.

The DFA state summarizes the information contained in the observed trace that is relevant to the satisfaction of the temporal specification. Hence, we can interpret it as a finite-memory representation of the progress made toward the task.
This representation is particularly useful for temporally extended tasks. If we consider different trajectories that lead the environment in the same physical state at a certain time, we will be able to identify the current correct stage of the temporal objective in each trajectory. Hence, the environment state and the automaton state can be considered jointly through an augmented representation

$$\tilde{s}_t = (s_t, q_t), \tag{2.46}$$

where s_t represents the physical configuration of the environment and q_t corresponds to the logical progress with respect to the LTLf specification. This representation makes the task-relevant history explicitly available to the learning process and it is the basis for the treatment of temporally extended objectives adopted in the framework presented in Chapter 3.

This construction has been formalized in the literature on NMRDP. In particular, in [4] it is shown that LTLf/LDLf non-Markovian rewards can be compiled into an equivalent Markovian MDP by augmenting the environment state with the states of the corresponding automata.

2.3 Hierarchical and Abstraction-Based Reinforcement Learning

RL problems can become particularly hard when they involve large state spaces, long decision horizons, or sparse reward signals. In these settings, learning exclusively from primitive interactions may require a large number of experiences before useful behaviors are discovered.
One of the main contributions in the literature is the Hierarchical Reinforcement Learning (HRL) framework. HRL extends the standard RL framework by introducing structure into the decision-making process. The core idea is to decompose a complex problem into simpler components or to represent decisions at different levels of abstraction, reducing the effective complexity faced by the learning agent.
Several approaches have been proposed to introduce hierarchy in RL. One of them is the MAXQ framework in which the original MDP is decomposed into a hierarchy of sub-tasks and the representation of the value function is performed through a corresponding hierarchical decomposition [8]. Instead, Hierarchies of Abstract Machines (HAMs) constrains the space of admissible policies through a hierarchy of finite-state machines that organize the execution of primitive actions and lower-level behaviors [11].
A completely different form of hierarchy is based on the concept of temporal abstraction. The main contribution is represented by the Options framework which extends the notion of primitive actions by introducing temporally extended courses of action, called options. Options are defined by an initiation set, an internal policy, and a termination condition [13]. An option can therefore remain active for several

real environment steps, allowing the agent to reason over behaviors that operate at different temporal scales.
Hierarchy can also be introduced through abstractions of the state and action spaces. In this case, multiple concrete configurations can be represented through a smaller set of abstract states or actions, allowing decision making to be performed on a simplified representation of the original problem. More recent works have investigated formal abstractions that preserve relevant properties of the original value function while compressing the decision space [1].
These approaches illustrate that hierarchy in RL may arise through different mechanisms, including task decomposition, temporal abstraction, and abstraction of the underlying decision space. The framework considered in this thesis primarily follows the latter perspective. In particular, it relies on abstract representations of the environment state space that support reasoning and planning at different levels of granularity. State abstraction and the construction of the corresponding abstract models are therefore introduced in the following subsection.

2.3.1 State Abstraction and Abstract Models

State abstraction techniques aim to reduce the complexity of a decision-making problem by grouping concrete states that are considered equivalent with respect to the information relevant for control [1]. Formally, a state abstraction can be modeled through a mapping function

$$\phi : \mathcal{S} \to \overline{\mathcal{S}} \tag{2.47}$$

where $\mathcal{S}$ is the original state space and $\overline{\mathcal{S}}$ is the abstract state space. Multiple concrete states may therefore be mapped to the same abstract state, producing a coarser (and hopefully simpler) representation of the environment.
The main advantage of this process is that reasoning can be performed over a smaller state space. However, the abstraction mechanism comes out with a trade-off between compactness and information preservation. In other words, a coarser representation can simplify planning and learning, but on the other hand, the excessive aggregation of real states into abstract ones may remove relevant information for the optimal decision making. For this reason, abstraction methods are carefully designed to preserve specific properties of the original MDP, such as transition structure, rewards, or value functions.
An abstract state representation can be used to construct an abstract model of the original decision process. Formally, given an MDP

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$$

an abstraction mapping induces a reduced model

$$\overline{\mathcal{M}} = \left\langle \overline{\mathcal{S}}, \overline{\mathcal{A}}, \overline{P}, \overline{R}, \gamma \right\rangle$$

where the states and, depending on the adopted abstraction, possibly also the actions correspond to higher-level representations of the original problem. The abstract transition and reward functions summarize the behavior of the concrete states represented by each abstract state.

Different abstraction methods impose different conditions on how states can be grouped. Some abstractions aim at preserving behavioral equivalence, while others focus on preserving value information or approximately maintaining the decision-relevant properties of the original model. Value-preserving state-action abstractions, for instance, seek compact representations in which relevant value relationships are maintained despite the reduction of the original decision space [1].
Abstract models are particularly useful when they retain enough information to support reasoning about the original task while substantially reducing its complexity. Planning can then be performed directly at the abstract level, producing information that can subsequently be transferred back to the concrete problem.

2.3.2 Abstract Planning and Reward Shaping

Once an abstract model of the original decision process has been constructed, planning techniques can be adopted directly on the reduced state space. Since the abstract model generally contains fewer states than the concrete MDP, standard dynamic programming methods can be applied at a lower computational cost to estimate the long-term desirability of abstract states.
Given an abstract MDP

$$\bar{\mathcal{M}} = \langle \bar{S}, \bar{A}, \bar{P}, \bar{R}, \gamma \rangle,$$

a value function

$$\bar{V} : \bar{S} \to \mathbb{R}$$

can be computed through the Value Iteration or Q-Learning, as introduced in Section 2.1.3 and Section 2.1.5. The resulting values provide high-level information about the structure of the decision problem and, in a goal-oriented setting, can indicate which abstract states are more promising toward the desired objective. The information obtained through abstract planning can be exploited to guide learning at a more concrete level. In particular, a concrete state $s \in S$ can be mapped to $\phi(s)$ and considering $\bar{V}(\phi(s))$ we got a higher-level estimation of the desirability about the region containing s.

One common mechanism for transferring such information to the learning agent is reward shaping. Reward shaping introduces a new contribution in the original reward function with an additional signal intended to guide the agent toward the goal. Formally, given an original reward R and a shaping term F, the modified reward can be expressed as

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{2.48}$$

The most adopted formulation of reward shaping is the Potential-Based Reward Shaping (PBRS) [10]. In PBRS, the shaping term is defined through a potential function Φ over the state space as

$$F(s, s') = \gamma \Phi(s') - \Phi(s) \tag{2.49}$$

In [10] it is shown that, under the assumptions of the original formulation, $F(s, s')$ preserves the optimal policies of the underlying MDP while modifying the reward landscape experienced during learning.

A natural choice adopted to exploit the information coming from the abstract levels is to set

$$\Phi(s) = \bar{V}(\alpha(s)).$$

In this way, the agent receives additional feedback reflecting high-level information obtained through planning, even when the original reward signal is sparse.
This idea has been investigated in abstraction-based reinforcement learning, where abstract models are used to generate heuristic or shaping information for the underlying learning problem [5][6].

2.3.3 Multi-Level Abstraction Hierarchies

The state-abstraction mechanism described in the previous subsection is not limited to a single abstract representation of the ground environment. Instead, multiple abstractions can be organized into a hierarchy, as formalized in [6], in which each level provides guidance to the level immediately below.
Let

$$\mathcal{M}^{(0)}, \mathcal{M}^{(1)}, \ldots, \mathcal{M}^{(L)}$$

denote a hierarchy of MDPS, where $\mathcal{M}^{(0)}$ corresponds to the most concrete level and each $\mathcal{M}^{(i+1)}$ is an abstraction built on $\mathcal{M}^{(i)}$. Therefore, the hierarchy a sequence of abstraction mappings between consecutive levels,

$$\mathcal{M}^{(0)} \to \mathcal{M}^{(1)} \to \cdots \to \mathcal{M}^{(L)}.$$

As the abstraction level increases, the corresponding model generally becomes more compact and provides a coarser representation of the underlying decision problem. Higher levels can therefore capture global structural information, while lower levels provide a more detailed description of the environment.
The crucial feature of this framework is that the information computed at a higher abstraction level can be exploited at the level below. In the framework proposed in [6] abstract models are solved in order to obtain value or heuristic information that can be transferred downward, as explained in the previous subsection.
Conceptually, the information flow can be represented as

$$\mathcal{M}^{(L)} \xrightarrow{\text{planning}} V^{(L)} \to \mathcal{M}^{(L-1)} \xrightarrow{\text{planning}} V^{(L-1)} \to \cdots \to \mathcal{M}^{(0)},$$

where the information obtained at each level contributes to the guidance of the level below.
Intuitively, coarser levels can provide information about long-range relationships between large regions of the environment, whereas finer abstractions can preserve more detailed information about local transitions. Hence, the hierarchy combines global and local information rather than relying on a single abstraction granularity.